PROJECT DOCUMENTATION / V1 CONCEPT

Issue Triage Assistant

Evidence-grounded decision support for internal audit issue triage and risk assessable unit scoping.

Purpose

Help auditors find relevant precedent, controls, and policy; prepare a cited recommendation; and make the final decision themselves. The application is designed to reduce repeated search and drafting work while leaving a reviewable evidence trail.

Product boundary

The assistant suggests. A human auditor approves, edits, or rejects. It never auto-approves, closes, creates, or files an issue. This concept build uses fictional synthetic data only.

DOCUMENT	SCOPE	DATA
Project overview and operating guide	Single-issue triage + RAU scoping	Synthetic fixtures only
Version 1 concept	Streamlit, LangChain, LangGraph, Pinecone, SQL	Prepared 30 October 2026

Prepared from the repository README, product specification, system prompt, and current application workflow.

01 / Problem and product

Audit issues can describe the same condition in different language. That makes precedent hard to find and can lead to inconsistent severity, incorrect control mapping, repeated records, and remediation work that is not reused. A recommendation without a source trail also slows reviewer validation.

What the assistant provides

CURRENT PAIN	ASSISTANT SUPPORT
Same issue appears under different wording	Semantic and keyword retrieval surface similar past issues.
Severity varies across auditors	A structured severity suggestion is grounded in precedent and the configured rubric.
Control mapping is missing or incorrect	Control documents are retrieved and cited; auditor hints guide search only.
Prior remediation is hard to locate	Relevant actions from past issues and policies inform a draft remediation plan.
Review basis is hard to verify	Each supported claim links to source and chunk identifiers plus an exact quote.

Intended users

- Auditor: supplies issue context, reviews sources, and owns disposition.
- Reviewer: checks the recommendation, source chunks, citations, and recorded decision.
- Operator: configures providers, persistence, identity, access controls, and deployment.

Non-goals

- No final severity decision and no evidence-sufficiency judgement.
- No automatic issue approval, closure, creation, filing, or external-system write-back.
- No real client names, audit records, or evidence in this concept environment.
- No multi-agent swarm; one explicit graph keeps retrieval and review steps inspectable.

02 / User workflows

Single-issue triage

The auditor enters a title and description, selects the synthetic client and audit, optionally adds context and a control hint, and identifies themselves for the review trail. The selected audit scopes duplicate screening. Similar precedent can come from other audits for the same synthetic client. Controls and policies are retrieved from that client library.

- Output: suggested severity (Critical, High, Medium, Low), root-cause category, control mapping, duplicate status, up to five similar issues and outcomes, remediation steps with generic owner roles and due-date suggestions, confidence, and citations.
- The reviewer may approve, edit, or reject. Rejection requires a reason; edited claims are citation-validated again.
- The app has an approved draft issues list. Selecting an issue title opens a detail dialog with the input, stored suggestion, similar issues, citations, and human decision.

RAU scoping

Scoping starts with a required synthetic client and RAU ID. Control, audit context, audit period, RAU name, and attributes refine the search when entered. Audit context does not restrict cross-audit precedent. A recognized year in the audit period limits relevant issue, control, and policy retrieval to matching audit IDs for that client. A two-year issue lookback is applied, and older matching issues are shown as excluded.

- Past issues are grouped by control, attribute, and root cause.
- The UI surfaces suggested testing actions, source issues, control-version context, recurring causes, and older excluded issues.
- Each cluster pauses for an auditor decision (approve, de-scope, or escalate) and a recorded reason.

03 / System design

The app combines deterministic retrieval and validation with structured model output and a durable human review checkpoint. Retrieval and citation diagnostics remain visible to support demonstration and review.

Request path

AUDITOR INPUT	FILTERED RETRIEVAL	SUGGESTION	VALIDATION + HUMAN REVIEW
Issue, synthetic client/audit, optional context and control hint	Pinecone semantic + local BM25 keyword results; metadata filters applied before ranking; RRF fusion	LangChain structured output using rubric, taxonomy, retrieved chunks, and optional approved pattern memory	Exact chunk/source/quote checks; one repair attempt; LangGraph interrupt; auditor approve/edit/reject

Core components

COMPONENT	RESPONSIBILITY
Streamlit (app.py)	Issue form, scoping interface, retrieval diagnostics, citations, chat, review controls, approved issue details.
LangChain	Prompting and typed structured model responses.
LangGraph	Single-issue retrieval/validation/review workflow and RAU scoping graph; durable interrupt/checkpoint for pending human review.
Pinecone	Vector index partitioned into past_issues, controls, and policies namespaces.
Local BM25 + RRF	Lexical matching over the small generated synthetic corpus; merges rankings with semantic results. RRF is a rank, not probability.
SQLAlchemy + SQLite/Postgres	Issues, runs, approvals, scoping packs/decisions, and append-only audit events. SQLite is for local learning; Postgres is the hosted path.
Optional Mem0	Auditor/client-scoped approved patterns only. No raw issue text, source chunks, or unapproved drafts.

Data flow and auditability

Synthetic CSV and Markdown definitions generate past-issue records and source documents. Controls are authored as Word documents and policies as PDFs; ingestion extracts and chunks their text for indexing. Runtime retrieval returns source metadata and chunk text, which the model uses to propose claims. The citation validator verifies that a cited chunk was retrieved and that the quote is exact. Review and retrieval events are recorded in SQL.

04 / Retrieval, citations, and confidence

Hybrid retrieval

For each relevant corpus, vector similarity retrieves semantically related wording while BM25 matches important terms. Metadata filters are applied before ranking. Reciprocal Rank Fusion combines the two ranked lists. The UI shows semantic score, keyword score, and RRF rank to make the hybrid behavior demonstrable.

- Exact-audit duplicate check: only records from the selected client and audit may support a duplicate warning.
- Similar precedent: other audits for the same client may inform triage, but are not called duplicates.
- RAU scoping: filters by required RAU and synthetic client; optional control and recognized audit year refine the search before ranking.
- Controls and policy are stored in their own namespaces and retrieved as supporting sources.

Citation validation

Supported fields carry source ID, chunk ID, and verbatim quoted text. Validation checks that the source/chunk was retrieved, the source type is suitable where required, and the quote exactly matches the chunk. One repair attempt is allowed. If claims cannot be validated or retrieval is weak, the result enters an explicit insufficient-evidence state and is not sent as a grounded recommendation to human approval.

A valid quote only proves that the text exists in the retrieved chunk. It does not prove the chunk semantically supports the claim; the auditor must assess relevance.

Confidence language

Confidence is a model-supported assessment based on relevance and agreement across retrieved sources, with citations. Pinecone similarity, keyword overlap, and RRF are retrieval diagnostics, not calibrated probabilities. Retrieval weakness is not an evidence-sufficiency conclusion.

05 / Human oversight and persistence

Single-issue decisions

After citation checks, LangGraph pauses for human action. Approve, edit, or reject is an explicit reviewer decision; the app does not take any downstream filing or closure action. Edits are revalidated before saving. Rejections include a reason.

ACTION	SYSTEM BEHAVIOR
Approve	Stores the approved suggestion, reviewer identity, reason, and decision time in the SQL audit record.
Edit	Allows changes to the draft; revalidates supported claims and stores the accepted version with review details.
Reject	Records the rejection and required reason; displays a denial confirmation. The issue remains unfixed in the external world.

RAU decisions

Each suggested scoping cluster requires an explicit auditor decision and reason. Cluster decisions are stored with the scoping run for later review. No cluster decision automatically creates or updates an audit issue.

Audit record and approved drafts

The SQL store retains issue inputs, run metadata, retrieval results, suggestions, reviewer decisions, scoping details, and audit events. Pending human reviews are checkpointed so they can be resumed after an app restart when a persistent checkpoint backend is configured. Approved or edited drafts appear in the UI history list; the title link opens the complete stored issue details and citations.

Optional memory

When configured, Mem0 may recall and store only a concise approved/corrected pattern scoped to auditor and synthetic client. Memory is terminology guidance, not evidence, and cannot substitute for current retrieved source chunks.

06 / Local operation and deployment

Local learning setup

Use Python 3.11-3.13, install the pinned direct dependencies, configure provider credentials in a local .env file, generate the synthetic source documents, ingest to a dedicated Pinecone index, and start Streamlit. Do not put secrets in source code, widgets, graph state, or logs.

STEP	COMMAND
Create virtual environment	python3.11 -m venv .venv (then activate it)
Install dependencies	pip install -r requirements.txt
Set local configuration	cp .env.example .env (then set required API keys)
Generate synthetic data	python scripts/generate_synthetic_data.py
Ingest the synthetic corpus	python -m src.issue_triage.ingestion.cli_ingest
Run the app	streamlit run app.py

Data and index contract

One Pinecone index uses cosine similarity with text-embedding-3-small at 1,536 dimensions (AWS us-east-1 default in the project). Three namespaces separate past issues, controls, and policies. Rebuild ingestion deletes the configured namespaces, so use it only with the dedicated concept index. SQLite is suitable for a local single-user learning project; Postgres is scaffolded for hosted use.

Production readiness gates

- Implement and verify identity-to-client/audit authorization on every retrieval, history, and persistence path. OIDC sign-in alone does not establish tenant authorization.
- Complete privacy, retention, encryption, secret management, backups, monitoring, incident response, vendor review, and database recovery/access controls.
- Generate and commit a complete platform-specific transitive dependency lock; direct dependency pins alone are not a resolved lock.
- Independently calibrate and evaluate retrieval thresholds, severity rubric, prompt behavior, and citation relevance using approved, labeled data.
- Validate migrations, deployment configuration, alerting, and operational runbooks before any real client data is introduced.

Current scope limits

The lexical index is a local BM25 scan over the small synthetic corpus and must be replaced or synchronized with a tenant-filtered lexical service at scale. The project has no real tenant data or organization-specific access policy, and its retrieval thresholds are concept calibration rather than a validated production model. Use synthetic examples only.

Repository references

README.md · docs/product_spec.md · docs/system_prompt.md · AGENTS.md · app.py · src/issue_triage/